DATABASES

NewSQL Visual Guide

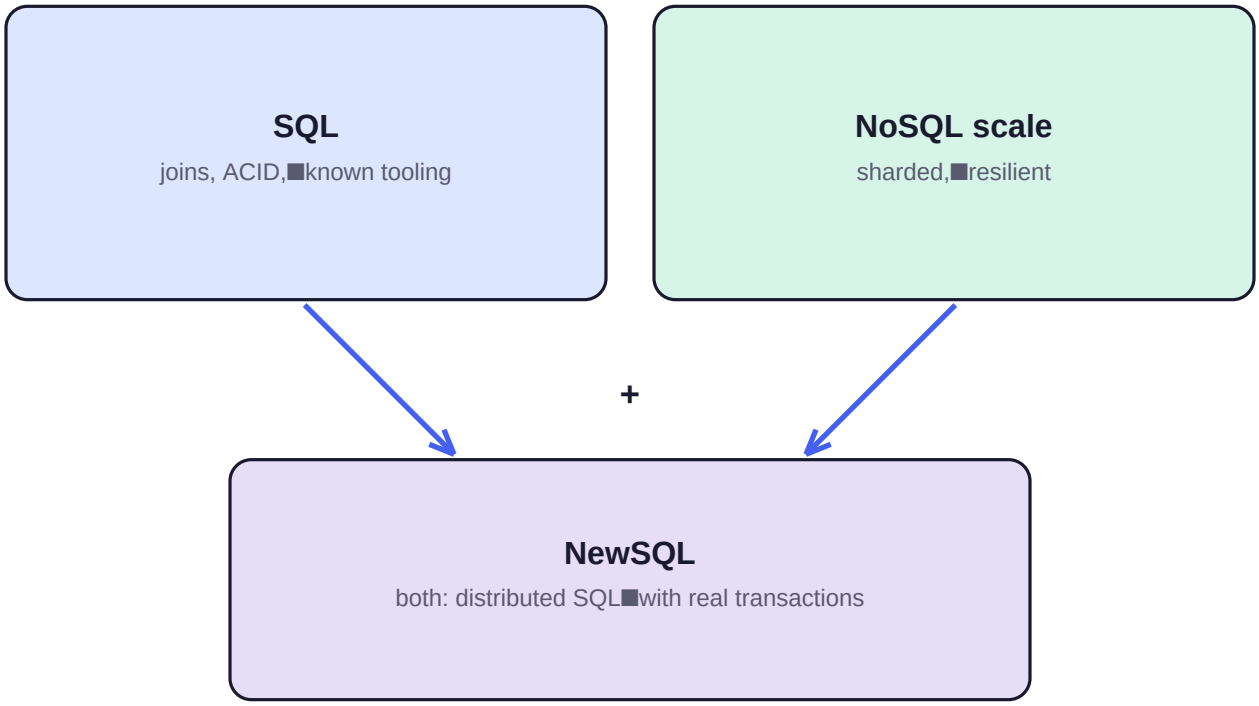
Spanner, CockroachDB --
relational guarantees at planetary scale.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The NewSQL Promise

Why not just use NoSQL?



STEP 2/6

Spanner & TrueTime

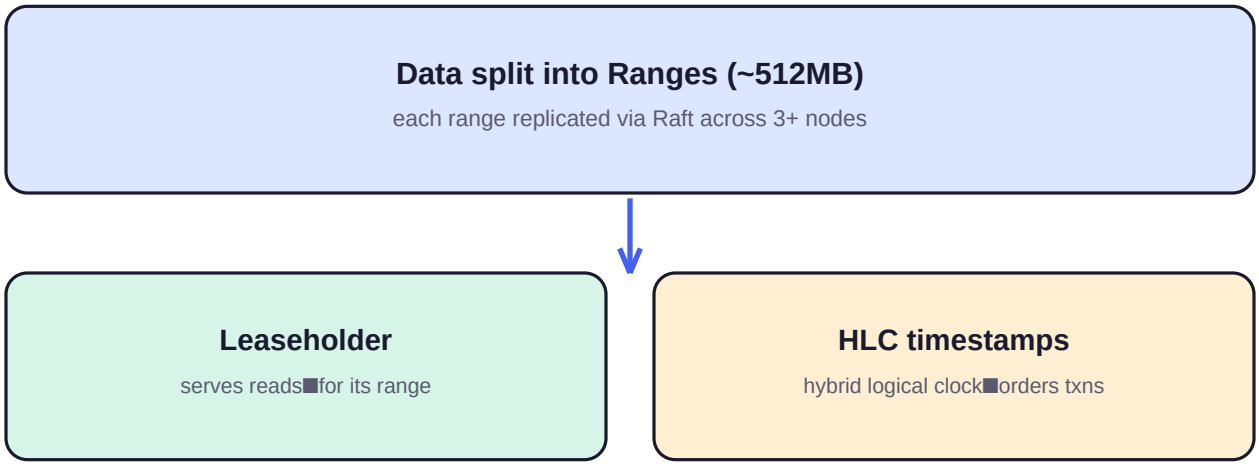
Globally consistent reads via synchronized clocks.

- TrueTime returns now as an INTERVAL [earliest, latest] -- uncertainty is explicit.
- Transactions commit-wait out the uncertainty: everyone agrees on order globally.
- Result: EXTERNAL CONSISTENCY -- the system behaves as if one machine, worldwide.

STEP 3/6

CockroachDB

Spanner's ideas, open source, no atomic clocks.

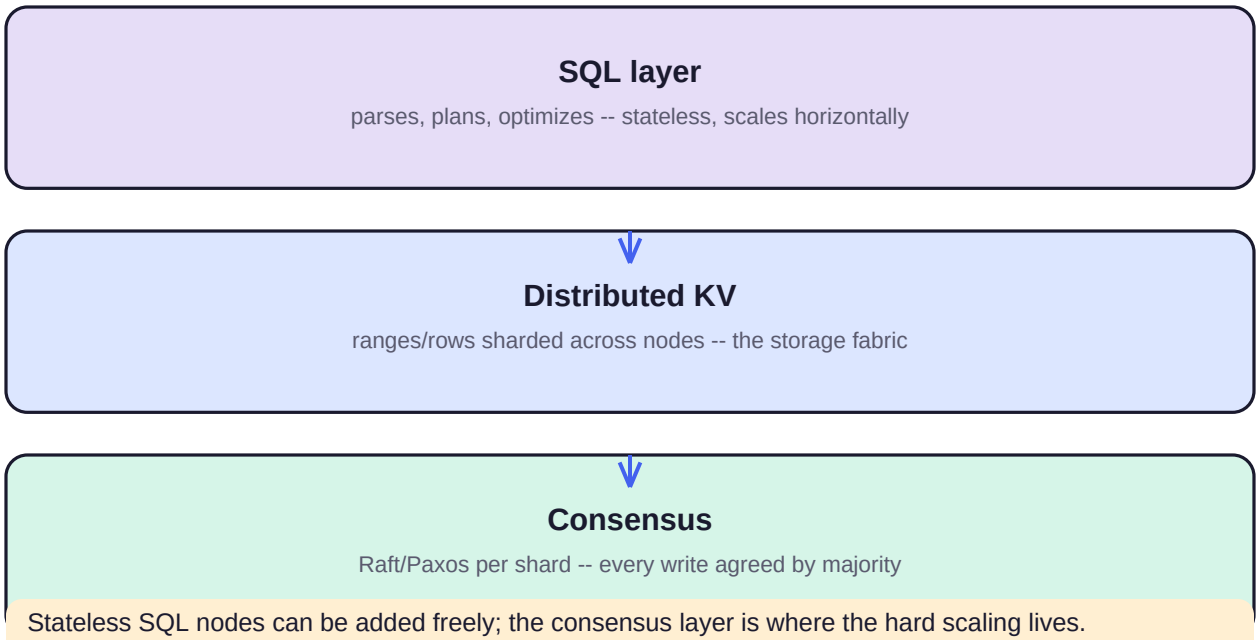


- Raft consensus per range: a majority ack makes a write durable.
- HLC (physical + logical time) orders transactions without TrueTime hardware.
- Node death: ranges re-replicate automatically -- self-healing by default.

STEP 4/6

Distributed SQL Architecture

Three layers, cleanly separated.



Stateless SQL nodes can be added freely; the consensus layer is where the hard scaling lives.

STEP 5/6

Trade-offs

Honest costs of planetary consistency.

Latency

consensus needs cross-node round trips -- single-digit ms at best

Operational complexity

multi-region clusters are not 'set and forget'

Cost

3x replication minimum -- durability is never free

Use NewSQL when you need global ACID (payments, inventory). Otherwise Postgres + replicas is simpler.

STEP ✓/✓

Key Takeaways

NewSQL -- Remember

- NewSQL keeps SQL semantics and ACID while sharding like NoSQL.
- Spanner's TrueTime turns clock uncertainty into global external consistency.
- CockroachDB replicates data ranges with Raft and orders transactions via hybrid logical clocks.
- Architecture: stateless SQL layer, distributed KV fabric, per-shard consensus.
- Costs are real: consensus latency, operational complexity, 3x replication -- use when global ACID pays.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev